MalScope

Automated Malware Analysis & Reporting Tool

Academic Technical Report

Project	MalScope-Analyzer
Domain	Cybersecurity / Malware Analysis
Tech Stack	Python, PyQt5, Google Gemini API, VirusTotal API
Date	May 2026

Team Members

#	English Name	Email	Section num
1	Khaled Ahmed Fathy Ahmed	Khaledhurtl115@gmail.com	1
2	AbdAlRaouf AbdAlKareem MohammedFares AlMasri	masriaboda12345@gmail.com	1
3	Karim Ahmed Melegy Mohamed Afifi	karim.melegy.afifi@gmail.com	1
4	Mahmoud Adel AbdAlSabour Mohammed	mahmoud.adel0221@gmail.com	2
5	Youssef Mohammed Mehrez Mohammed	yousefmohamedmehrez@gmail.com	2

Abstract

MalScope is a modular, automated malware analysis platform designed to simulate a real-world Security Operations Center (SOC) analysis pipeline. The system integrates static analysis, dynamic behavioral analysis, a weighted risk scoring engine, AI-driven threat explanation using Google Gemini, and automated PDF report generation into a cohesive desktop application built with Python and PyQt5.

This report provides a comprehensive academic examination of MalScope's system architecture, module design, implementation details, security methodology, and collaborative development practices. It further analyzes the scoring logic and normalization techniques employed, the threat intelligence integrations (VirusTotal API, Hybrid Analysis sandbox, and IP geolocation), the malware attribution and fingerprinting capabilities, error handling and resilience strategies, and a forward-looking roadmap for AI heuristics and machine learning enhancements.

MalScope demonstrates how a multi-layered cybersecurity tool can be built with clean modular separation of concerns, standard JSON-based inter-module communication, and production-grade design patterns suitable for both academic evaluation and real-world SOC deployment.

Table of Contents

1. Introduction & Project Overview

In modern cybersecurity operations, the ability to rapidly triage and classify suspicious files is critical to minimizing damage from malware incidents. Security Operations Center (SOC) analysts routinely encounter hundreds of potentially malicious artifacts per day, requiring efficient tooling that can automate the most time-consuming analysis steps while delivering actionable intelligence.

MalScope addresses this challenge by providing a fully automated malware analysis pipeline packaged as an intuitive desktop application. The tool is designed around the principle of modular independence: each analytical capability is implemented as a self-contained module communicating through well-defined JSON interfaces, orchestrated by a central controller, and presented through a responsive PyQt5 graphical user interface.

1.1 Project Goals

- Build a real-world malware analysis pipeline that mirrors professional SOC workflows
- Simulate multi-stage triage: static profiling, dynamic behavioral observation, and AI-assisted interpretation
- Ensure modular cybersecurity system design for independent development and testability
- Generate readable, professional-grade security reports suitable for management and technical audiences
- Provide an extensible foundation for future machine learning and heuristic enhancements

1.2 Technology Stack

Component	Technology	Purpose
GUI	PyQt5	Desktop UI with signal/slot architecture
Backend Logic	Python 3.x	Core analysis, orchestration
PE Analysis	pefile	Windows executable inspection
AI Analysis	Google Gemini API	LLM-powered threat explanation
Threat Intel	VirusTotal API v3	Global malware signature database
Sandbox	Hybrid Analysis API	Cloud-based detonation
Reporting	fpdf2	Automated PDF generation
Secrets Mgmt	python-dotenv	Secure API key management

2. System Architecture & Design

MalScope follows a layered pipeline architecture where each processing stage is cleanly separated from the others. The architecture can be summarized as a directed acyclic graph of processing stages, each consuming the output of the previous and producing a standardized JSON structure for consumption by the next.

2.1 High-Level Pipeline

GUI	→	Orchestrator	→	Static	→	Dynamic	→	AI +Reporting

This pipeline reflects a classic defense-in-depth triage model used in professional malware analysis: fast static checks first, followed by more expensive behavioral analysis, culminating in AI-assisted interpretation and human-readable reporting.

2.2 Design Principles

Modularity & Independence

Every module in MalScope is designed to operate independently of all other modules. No module imports from another module's internal logic. All inter-module communication is mediated exclusively by the Orchestrator, which acts as the system's controller and data bus. This design enables individual modules to be developed, tested, and replaced without impacting the broader system.

Signal-Driven UI Architecture

The PyQt5 GUI communicates with the backend Orchestrator exclusively through Qt's signal/slot mechanism via a centralized Signal Hub. This decouples the presentation layer from the business logic layer, ensuring that long-running analysis operations execute in background QThread workers without blocking the user interface. The Orchestrator registers handlers for signals such as request_scan, request_stop, and request_report, and emits progress events such as scan_progress, file_result_ready, and scan_completed back to the GUI.

Standard JSON Contract

Every module must produce output conforming to a pre-defined JSON schema. This contract-based design ensures that the Orchestrator can reliably consume outputs from any module regardless of implementation details, and that modules can be independently unit-tested by validating their JSON output against the schema.

2.3 Data Flow

1. User selects a folder via the GUI file picker
2. GUI emits request_scan signal with folder path and scan mode
3. Orchestrator spawns a ScanWorker in a dedicated QThread
4. ScanWorker iterates over all files in the directory
5. For each file: static analysis is executed, then dynamic analysis (if Full Analysis mode)
6. VirusTotal detection ratio and dynamic behavioral score are normalized to a 0-10 scale
7. Weighted risk score is computed: (Static * 0.6) + (Dynamic * 0.4)
8. LLM Analyzer generates a human-readable threat summary
9. Result dictionary is emitted to the GUI via file_result_ready signal
10. Upon completion, scan_completed signal triggers summary statistics update
11. User may request PDF report generation for selected files

3. Module Breakdown

MalScope comprises nine primary modules, each with clearly defined input/output contracts. The following subsections describe the responsibilities, data formats, and implementation details of each module.

3.1 GUI Module (PyQt5) — gui/

The GUI module provides the user-facing interface through which all system interactions are initiated. Built with PyQt5, it implements a multi-tab interface exposing the following views: a scan control panel, a results table displaying file name, hash, verdict, and risk score, a dynamic analysis detail tab, an AI Insights tab, and a report viewer.

The GUI module acts as a pure input/output layer. It emits signals to the Orchestrator and renders data received back from the Orchestrator. It performs no analysis logic of its own.

Signal	Description
request_scan	Emitted when user clicks Scan; carries folder path and mode
request_stop	Emitted when user clicks Stop
request_report	Emitted when user requests PDF report generation
file_result_ready	Received from Orchestrator; updates results table
scan_progress	Received from Orchestrator; updates progress bar
scan_completed	Received from Orchestrator; displays summary statistics

3.2 Core Orchestrator — core/orchestrator.py

The Orchestrator is the central nervous system of MalScope. It registers as a listener on the Signal Hub and responds to GUI requests by spawning background workers. Its primary

responsibilities are: invoking analysis modules in the correct sequence, aggregating results, computing the final risk score, calling the LLM Analyzer, and delegating report generation.

The Orchestrator implements a ScanWorker class that runs in a dedicated QThread to prevent UI blocking. The worker iterates over all files in the selected directory and executes the full analysis pipeline for each file. Results are emitted back to the GUI after each file completes, enabling real-time table updates rather than batch completion.

The Orchestrator enforces the 60/40 weighted scoring formula and determines the final verdict according to the following thresholds:

Score Range	Verdict	Severity
0.0 – 2.9		Low
3.0 – 6.9	**Suspicious**	Medium
7.0 – 10.0		High

3.3 Static Analysis Engine — analysis/static/

The Static Analysis Engine performs non-executable inspection of files to extract structural properties, cryptographic signatures, and embedded network indicators. It is composed of four sub-modules working in concert under the coordination of static_analyzer.py.

static_analyzer.py — Core Aggregator

Computes five cryptographic hashes (MD5, SHA-1, SHA-256, SHA-512, and IMPHASH) and coordinates the output of all sub-modules into a single unified JSON report. The IMPHASH (Import Hash) is particularly valuable for malware attribution as it uniquely fingerprints the import table of a PE executable.

pe_analysis.py — PE Header Dissection

Uses the industry-standard pefile library to extract: the Import Address Table (IAT) and Export Address Table (EAT) to identify dangerous API usage; Authenticode digital signature status; section metadata including name, entropy, virtual size, and raw size; compile timestamp; and embedded resource types and counts. Packer detection uses signature-based heuristics matching known packer names (UPX, Enigma, Themida, ASPack, MPRESS) against section names. High entropy (>7.0) in any section is flagged as a suspicious indicator of packed or encrypted code.

strings_analyzer.py — IOC Extraction

Extracts both ASCII and Unicode strings (minimum 4 characters) and applies regex-based classification to produce a structured priority_strings output containing URLs, IP addresses, file paths, and registry keys. Noise reduction is achieved through length thresholds (>8 chars), alphanumeric ratio filters, and dynamic blacklists excluding known library artifacts. Extracted IP addresses are automatically enriched with geolocation data (country and ISP) via the ip-api.com service.

vt_client.py — Threat Intelligence

Submits the computed SHA-256 hash to the VirusTotal API v3 and retrieves the aggregate detection ratio from the global antivirus engine database. The result includes the count of malicious, suspicious, and undetected verdicts out of the total number of engines that scanned the file, as well as a permalink to the full VirusTotal report.

3.4 Dynamic Analysis Engine — analysis/dynamic/

The Dynamic Analysis Engine performs behavioral analysis of suspicious files to predict and detect malicious runtime actions. Critically, the engine operates in safe mode by default: it does not execute files directly on the host system. Instead, it performs predictive behavioral analysis based on binary content inspection.

dynamic_analyzer.py — Main Entry Point

Orchestrates all dynamic analysis steps: string extraction for behavioral prediction, PE import analysis for dangerous API detection, and sandbox submission. Results from all sub-steps are merged and scored by behavior_parser.py.

behavior_parser.py — Behavioral Scoring

Parses and normalizes behavioral data from all dynamic sub-modules into a standardized output structure. It assigns scores to each detected behavioral indicator and aggregates them into a final dynamic score on a 0-10 scale. Indicators scored include: spawned shell processes (cmd.exe, powershell.exe), direct IP-based outbound connections (C2 channel indicators), registry persistence key modifications, file operations in sensitive directories, and detection of high-risk Windows API imports (CreateRemoteThread, VirtualAllocEx, InternetConnectA).

sandbox_client.py — Hybrid Analysis Integration

When configured with a Hybrid Analysis API key, this module submits the file to the cloud sandbox for actual detonation in an isolated VM environment. The sandbox provides verified process activity, network connections, registry changes, and file system modifications that corroborate or extend the local behavioral predictions.

3.5 Scoring Engine — analysis/scoring/

The Scoring Engine implements a weighted combination model that produces the final risk score from static and dynamic sub-scores. The formula is: Final Score = (Static Score x 0.6) + (Dynamic Score x 0.4). This 60/40 weighting reflects the higher reliability and broader coverage of static analysis data (particularly VirusTotal's aggregate of 75+ antivirus engines) relative to the predictive nature of the local behavioral analysis.

Both input scores are normalized to the 0-10 scale prior to combination. The static score is derived from the VirusTotal detection ratio (malicious detections / total engines) multiplied by 10. The dynamic score is produced directly by behavior_parser.py's weighted indicator aggregation.

3.6 AI / LLM Module — ai/llm_analyzer.py

The AI module leverages Google's Gemini large language model to translate the raw JSON analysis results into a structured, human-readable threat intelligence narrative targeted at SOC analysts.

The module employs careful prompt engineering to constrain the model's output to a specific format containing: a Threat Summary contextualizing the verdict, a bulleted list of Suspicious Indicators correlating static and dynamic findings, and Recommended Response steps appropriate to the severity. The system prompt explicitly instructs the model to avoid hallucination by grounding its analysis exclusively in the provided JSON data.

The module applies post-processing to strip markdown formatting characters (which are incompatible with the PyQt5 plain text display and the fpdf-based PDF generator), and handles API failures gracefully by returning a descriptive fallback message rather than crashing the pipeline.

3.7 PDF Report Generator — reports/pdf_generator.py

The Report Generator uses the fpdf2 library to produce structured PDF reports for each analyzed file. Reports are organized into three sections: an Executive Summary containing the target filename, scan date, final verdict (color-coded by severity), and risk score; an AI Threat Intelligence Summary containing the LLM-generated narrative; and a Technical Data section listing all static and dynamic analysis findings.

The generator includes a Latin-1 encoding safety filter that strips unsupported Unicode characters (such as emoji) that would crash the PDF renderer. Reports are saved to a generated_reports/ directory with filenames derived from the analyzed file's name.

4. Implementation Details

4.1 Entry Point — main.py

The application entry point initializes the QApplication instance, applies the global stylesheet via build_global_stylesheet(), instantiates the MainWindow, and wires the Orchestrator to the window's Signal Hub. A reference to the Orchestrator is stored on the window object to prevent garbage collection by Python's reference counting system — a subtle but critical requirement in PyQt5 applications where Python and C++ object lifetimes interact.

4.2 Threading Model

Long-running analysis operations (which may take several seconds per file for VirusTotal API calls and sandbox submissions) are executed in QThread worker objects to prevent UI freezing. The ScanWorker class is instantiated fresh for each scan session and moved to a new QThread via moveToThread(). Thread lifecycle management uses Qt's built-in finished signal connections to ensure proper cleanup: the worker emits finished upon completion, which triggers thread.quit(), and both worker and thread are marked for deletion via deleteLater(). The Orchestrator's _cleanup_thread slot nullifies the Python references after the C++ objects are destroyed, preventing RuntimeError on stale object access.

4.3 Scan Modes

Mode	Modules Executed	Use Case
Full Analysis	Static + Dynamic + AI	Comprehensive malware triage
Quick Scan	Static + AI (no dynamic)	Rapid initial screening
Static Only	Static only (no AI)	Hash verification, IOC extraction

4.4 File Discovery

The ScanWorker's _get_files() method uses os.listdir() to enumerate all regular files in the selected directory. The implementation includes a graceful fallback to a set of mock filenames when the directory cannot be read, enabling UI testing without real file system access. In production, os.walk() would be the preferred approach for recursive subdirectory traversal.

4.5 Report Generation Flow

The generate_report() slot in the Orchestrator accepts a scope identifier and a list of file result objects. It iterates over each result, extracts the AI explanation text, calls ReportGenerator.generate_pdf(), and tracks progress via the report_progress signal. Both single-file and batch reporting are supported, with the final output path emitted via report_completed for the GUI to display.

5. Security Methodology

MalScope's analysis methodology is structured around the industry-standard multi-stage malware triage model used in professional SOC environments. The following principles guide the security analysis approach.

5.1 Defense-in-Depth Triage

MalScope implements a three-layer analysis model. The first layer, Static Analysis, provides fast, safe, signature-based and structural inspection with no file execution risk. The second layer, Dynamic Analysis, provides behavioral prediction through binary content analysis and optional cloud sandbox detonation. The third layer, AI-Assisted Interpretation, provides contextual correlation of findings and actionable recommendations. Each layer compensates for the blind spots of the previous: static analysis may miss obfuscated or polymorphic code that behavioral analysis can detect; behavioral analysis may produce false positives that the AI layer can contextualize.

5.2 Safe-Mode Analysis

A critical design decision is that all local analysis (static and the local dynamic component) is performed without executing the suspicious file on the host system. Files are read as binary data and analyzed through structural parsing, string extraction, entropy calculation, and import table inspection. The only mode that involves execution is the optional Hybrid Analysis sandbox submission, which occurs in an isolated cloud VM environment.

5.3 Indicators of Compromise (IOC) Framework

MalScope systematically extracts and categorizes IOCs across multiple dimensions:

- Cryptographic IOCs: Five-hash fingerprinting (MD5, SHA-1, SHA-256, SHA-512, IMPHASH) for file identity verification and cross-reference with threat intelligence databases
- Network IOCs: URLs and IP addresses extracted from binary strings and enriched with geolocation data for threat actor attribution
- Host-Based IOCs: File paths in sensitive directories, registry keys for persistence mechanisms, and dangerous Windows API imports
- Behavioral IOCs: Predicted process spawning, C2 communication patterns, and registry modification for persistence

5.4 Verdict Classification

The three-tier verdict classification (Clean, Suspicious, Malicious) maps to standard industry threat severity levels and is designed to guide analyst response. A Suspicious verdict intentionally avoids definitive classification to reflect analytical uncertainty and prompt further investigation rather than automated remediation.

6. Scoring Logic & Normalization

6.1 Static Score Derivation

The static score is derived from VirusTotal detection results using the following normalization formula:

```
Static Score = (malicious_detections / total_engines) x 10.0
```

This formula normalizes the raw detection count to the 0-10 scale regardless of how many engines participated in the scan (which varies by subscription tier and file type). For example, 63 detections out of 75 total engines yields a static score of 8.4, which independently maps to a Malicious verdict.

In the absence of a VirusTotal result (API key not configured, rate limit exceeded, or network error), the static score defaults to 0.0 and the dynamic score carries full weight.

6.2 Dynamic Score Derivation

The dynamic score is computed by behavior_parser.py's weighted indicator aggregation. Each detected behavioral indicator contributes a fixed score increment, with the total capped at 10.0. Examples of indicator weights include:

Behavioral Indicator	Score Weight	Rationale
Suspicious API imports (CreateRemoteThread, VirtualAllocEx)	2.0 – 3.0	Code injection capability
Direct IP outbound connection (no DNS)	1.5 – 2.0	C2 channel indicator
Registry Run key modification	1.5	Persistence mechanism
Shell process spawning (cmd.exe, powershell.exe)	1.0 – 1.5	Command execution capability
File operation in sensitive directory (System32, Temp)	1.0	System file manipulation
High entropy section (>7.0) — packing/encryption	0.5 – 1.0	Obfuscation indicator

6.3 Weighted Combination Formula

```
Final Score = (Static Score x 0.6) + (Dynamic Score x 0.4)
```

The 60/40 weighting reflects the following analytical considerations: VirusTotal aggregates the verdict of 75+ antivirus engines representing decades of signature research, providing a high-confidence ground truth for known malware. Dynamic analysis, while valuable for detecting novel or obfuscated samples, is implemented in predictive mode (not live execution) in the local component, introducing inherent uncertainty that warrants lower weighting.

6.4 Score Rounding

Both static and dynamic sub-scores, as well as the final combined score, are rounded to one decimal place using Python's round() function. This rounding is applied before storage in the result dictionary to ensure GUI display consistency across all views (results table, detail pane, and PDF report).

7. Threat Intelligence Integration

7.1 VirusTotal API v3

VirusTotal integration is implemented in vt_client.py and uses the SHA-256 hash of the analyzed file as the lookup key. The integration queries the /api/v3/files/{hash} endpoint and extracts the last_analysis_stats object containing the count of malicious, suspicious, harmless, and undetected engine verdicts. The API key is loaded from a .env file using python-dotenv, ensuring credentials are never hardcoded in source control.

The integration handles the case where a file has never been submitted to VirusTotal (HTTP 404 response) by returning a not_found status rather than raising an exception, allowing the pipeline to continue with a static score of 0.

7.2 IP Geolocation — ip-api.com

All IP addresses extracted from binary strings are automatically enriched with geographic metadata using the free ip-api.com REST API. The enrichment provides country and ISP information, enabling rapid threat actor attribution. For example, an outbound connection to an IP resolved to a known hosting provider in a high-risk jurisdiction significantly increases the analytical confidence of a Malicious verdict. Private and reserved IP ranges are identified as Unknown/Local without API calls.

7.3 Hybrid Analysis Sandbox

The sandbox_client.py module integrates with Hybrid Analysis (a service by CrowdStrike) to submit files for cloud-based detonation in isolated Windows VM environments. This provides ground-truth behavioral data unavailable through local static inspection: actual process trees, real network connections, live registry changes, and file system modifications. The sandbox integration is optional and configured via an API key. When unavailable, the module returns a structured error response and the pipeline falls back to local behavioral prediction without interruption.

7.4 Secrets Management

All external API keys (VirusTotal, Gemini, Hybrid Analysis) are managed through environment variables loaded from a .env file at runtime. The .env file is explicitly excluded from version control via .gitignore. This follows the 12-factor application configuration principle, ensuring that credentials are never exposed in source code repositories and can be rotated independently of application deployments.

8. Malware Attribution & Fingerprinting

8.1 Five-Hash Fingerprinting

MalScope computes five distinct cryptographic hashes for each analyzed file, each serving a specific analytical purpose:

Hash	Algorithm	Analytical Use
MD5	128-bit	Legacy database lookup; fast triage against known malware hashes
SHA-1	160-bit	Improved collision resistance over MD5; widely supported in threat intel feeds
SHA-256	256-bit	Primary VirusTotal lookup key; current industry standard for file identity
SHA-512	512-bit	High-security verification; tamper detection in forensic contexts
IMPHASH	PE-specific	Fingerprints malware families by import table structure; robust to recompilation

8.2 IMPHASH for Family Attribution

The Import Hash (IMPHASH) is a particularly powerful technique for malware family attribution. It computes an MD5 hash of the ordered list of imported DLL names and function names, normalized to lowercase. Because members of the same malware family typically share the same import table structure (even across versions), IMPHASH allows analysts to cluster related samples and attribute new samples to known families without relying on exact file hash matches. This is especially effective against simple recompilation or minor code modification techniques used to evade signature-based detection.

8.3 Section Entropy Analysis

Shannon entropy measurement of PE sections provides a quantitative indicator of data randomness. Legitimate executable sections typically exhibit entropy values between 4.0 and 6.5. Values above 7.0 strongly suggest packed or encrypted content, a common technique used by malware to evade string-based static detection. MalScope flags all sections with entropy above 7.0 as suspicious indicators in the static analysis output.

8.4 Compile Timestamp

The TimeDateStamp field extracted from the PE header provides an estimated compilation date for the binary. While this timestamp can be forged by malware authors, it remains a useful data point for attribution: timestamps far in the past (e.g., Windows XP era) on recently active malware suggest timestamp manipulation, while consistent timestamps across a malware family's samples can confirm common tooling.

9. Risk Mitigation & Error Handling

9.1 Module-Level Exception Isolation

Each module call within the ScanWorker pipeline is wrapped in a try/except block. If any module raises an exception (network timeout, malformed binary, API rate limit, missing dependency), the exception is caught, logged to the GUI's log panel via the log_message signal, and the pipeline continues with an empty result for that module. This isolation ensures that a failure in one module (e.g., VirusTotal API downtime) does not abort analysis for the remaining modules or files.

9.2 Graceful Degradation

MalScope is designed to produce useful (if partial) results even when optional components are unavailable. VirusTotal, Gemini AI, and Hybrid Analysis are all optional integrations — the tool degrades gracefully to local analysis when these services are unavailable. The scoring engine handles missing sub-scores by defaulting to 0.0, ensuring the weighted formula always produces a valid output.

9.3 Thread Safety

All GUI updates are performed exclusively through Qt signals emitted from the worker thread and consumed in the main thread. Direct GUI manipulation from background threads is prohibited, as this is undefined behavior in Qt. The signal/slot mechanism automatically marshals cross-thread calls through Qt's event queue, ensuring thread-safe UI updates.

9.4 PDF Encoding Safety

The PDF generator applies a Latin-1 encoding filter to all AI-generated text before rendering. This prevents crashes caused by Unicode characters outside the Latin-1 range (such as emoji, CJK characters, or special symbols) that the fpdf2 core font renderer does not support. Characters that cannot be encoded are replaced with a substitution character rather than raising an exception.

9.5 Worker Lifecycle Management

The ScanWorker exposes a stop() method that sets an is_running flag to False. The main scanning loop checks this flag between files and between pipeline stages. When stop() is called (via the request_stop signal), the worker completes its current atomic operation, observes the flag, and terminates cleanly. This avoids the use of thread termination (QThread.terminate()), which is unsafe and can leave shared resources in an inconsistent state.

10. Team Collaboration Rules

MalScope was developed as a collaborative team project, and its architecture explicitly enforces collaboration-friendly development practices. The following rules govern how team members contribute modules to the system.

10.1 Module Independence

No module may import from another module's internal implementation. All inter-module communication is mediated by the Orchestrator. This rule eliminates circular dependencies, enables parallel development by multiple team members, and ensures that any module can be replaced or upgraded without impacting others.

10.2 Standard Output Format

Every module must return results as a Python dictionary serializable to JSON. The output schema for each module is defined in its README.md and must be strictly adhered to. The Orchestrator validates outputs by key access, so schema violations result in KeyError exceptions that are caught and logged.

10.3 Per-Module README Documentation

Each developer is responsible for maintaining a README.md in their module's directory documenting: what the module does, the expected input format, the output JSON schema with field descriptions, and at least one example output for both clean and malicious files. This

documentation is the integration contract between module developers and the Orchestrator developer.

10.4 No Black-Box Code

All module functions must have clear names, docstrings describing their purpose, and clean separation between data-fetching, data-processing, and data-formatting responsibilities. Monolithic functions that combine multiple concerns are prohibited. This rule ensures that any team member can understand, debug, and extend any module.

10.5 Orchestrator Authority

Only the Orchestrator is permitted to call modules, combine results, and determine the final verdict. Modules must not implement scoring or verdict logic — they produce raw data outputs, and the Orchestrator applies the scoring model. This ensures consistent verdict criteria across all analysis paths.

10.6 Integration Testing Checklist

Before a module is considered complete and ready for integration, it must pass the following independent test checklist:

- Input accepted: module processes the expected input without exceptions
- Output format correct: returned dictionary matches the documented schema
- No external dependencies: module does not import from other MalScope modules
- JSON serializable: output can be passed through json.dumps() without error

11. Future Roadmap & AI Heuristics

11.1 Machine Learning Classification

The project structure includes a models/ directory and a placeholder ml_model.pkl file, signaling the intended integration of a machine learning classifier as an optional alternative or complement to the rule-based scoring engine. A trained classification model (Random Forest or Gradient Boosting) could learn from historical VirusTotal verdicts and behavioral features to produce probability-calibrated malicious/clean predictions, particularly effective for novel malware families not yet in signature databases.

11.2 Enhanced AI Heuristics

The current LLM integration uses a single-pass prompt-response model. A more sophisticated implementation could employ multi-turn reasoning: first asking the model to identify the most anomalous indicators, then asking it to hypothesize potential attack chains, and finally requesting a confidence-scored verdict. This chain-of-thought prompting approach has demonstrated improved accuracy on complex analytical tasks compared to single-shot prompting.

11.3 YARA Rule Integration

Integrating the YARA pattern matching framework would enable detection of known malware families through custom signature rules without API calls. The static analysis engine could scan file contents against a maintained YARA rule database, providing offline detection capability and enabling analysts to contribute custom detection logic.

11.4 Network Traffic Analysis

A future PCAP analysis module could process network capture files alongside executable samples, enabling correlation between binary IOCs (extracted IP addresses and URLs) and actual observed network behavior. Integration with tools such as Zeek or Suricata would provide protocol-level behavioral signatures.

11.5 Threat Intelligence Platform Integration

Integration with MISP (Malware Information Sharing Platform) or OpenCTI would enable MalScope to both consume shared threat intelligence (enriching analysis with community-sourced IOC context) and contribute findings back to the community, transforming the tool from a standalone analyzer into a node in a collaborative threat intelligence network.

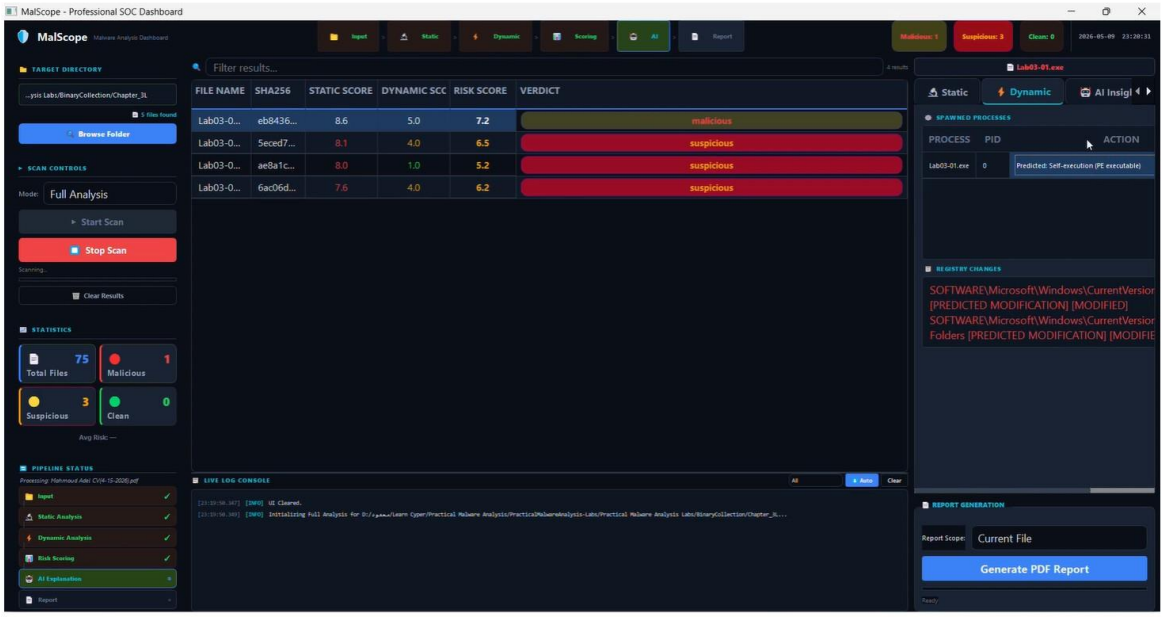
11.6 Sandbox Expansion

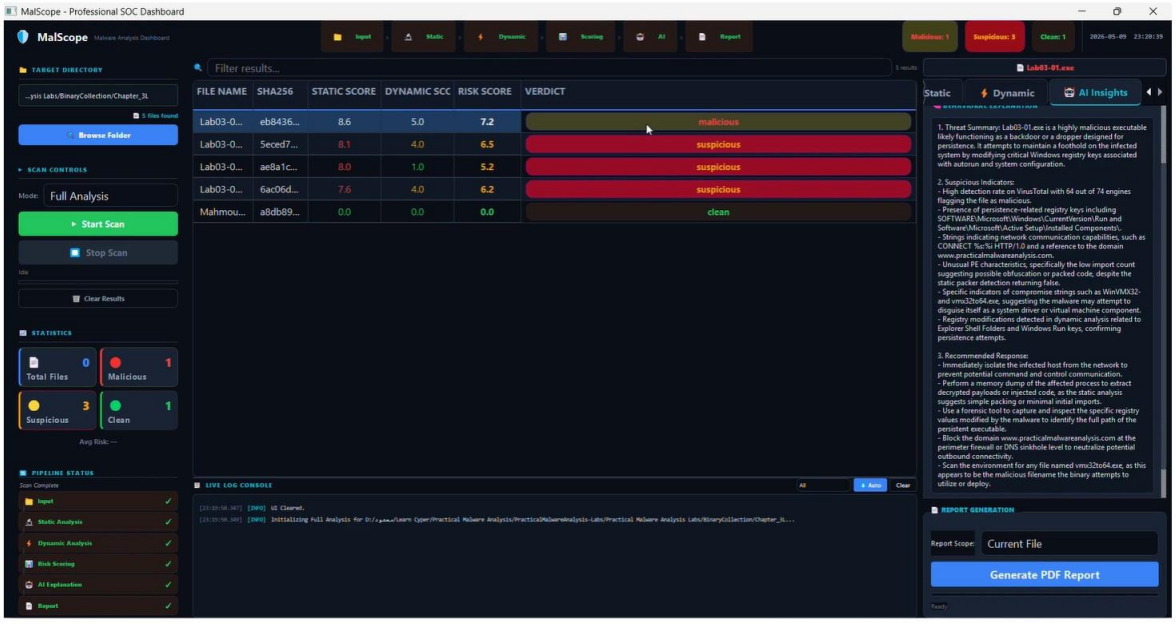
Beyond Hybrid Analysis, future versions could support additional sandbox providers including Any.run, Joe Sandbox, and locally-deployed open-source sandboxes such as Cuckoo or CAPE. A sandbox abstraction layer would allow the sandbox_client.py to be configured with any provider through a common interface.

11.7 Real-Time Dynamic Analysis

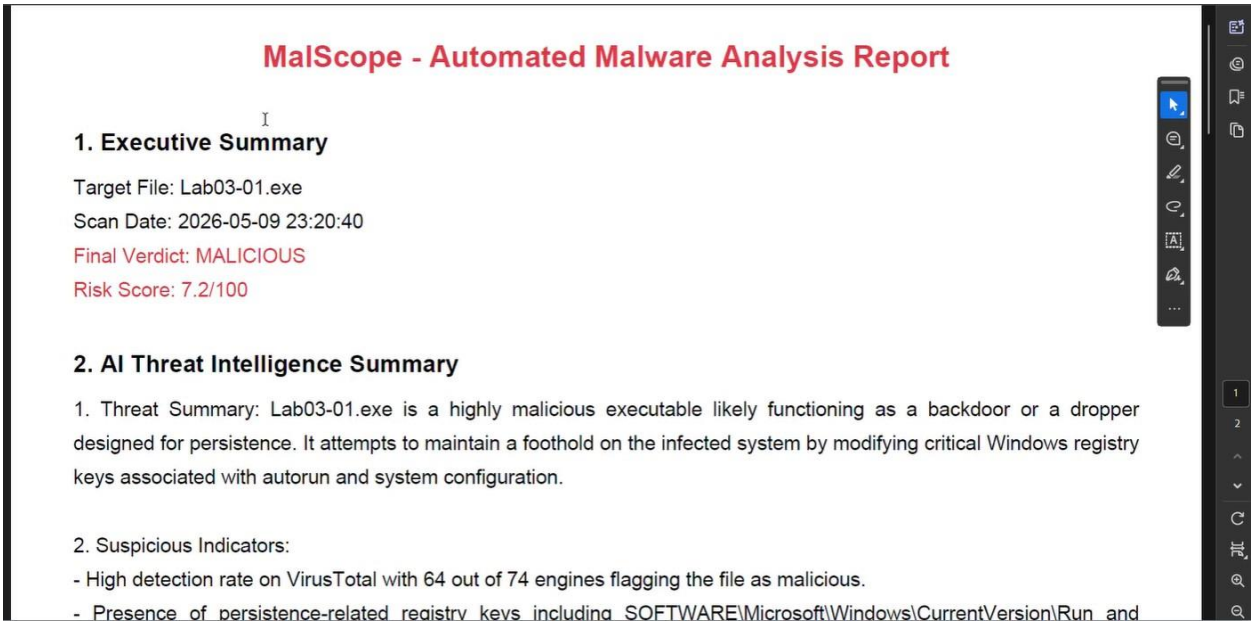
The most significant capability enhancement would be the implementation of true local dynamic analysis using Windows API hooking (via libraries such as frida) or kernel-level monitoring (via ETW event tracing). This would provide actual runtime behavioral data rather than predictions, significantly improving detection accuracy for heavily obfuscated or delayed-execution malware.

12. Screen Output of run programm





Output Report:



MalScope - Automated Malware Analysis Report

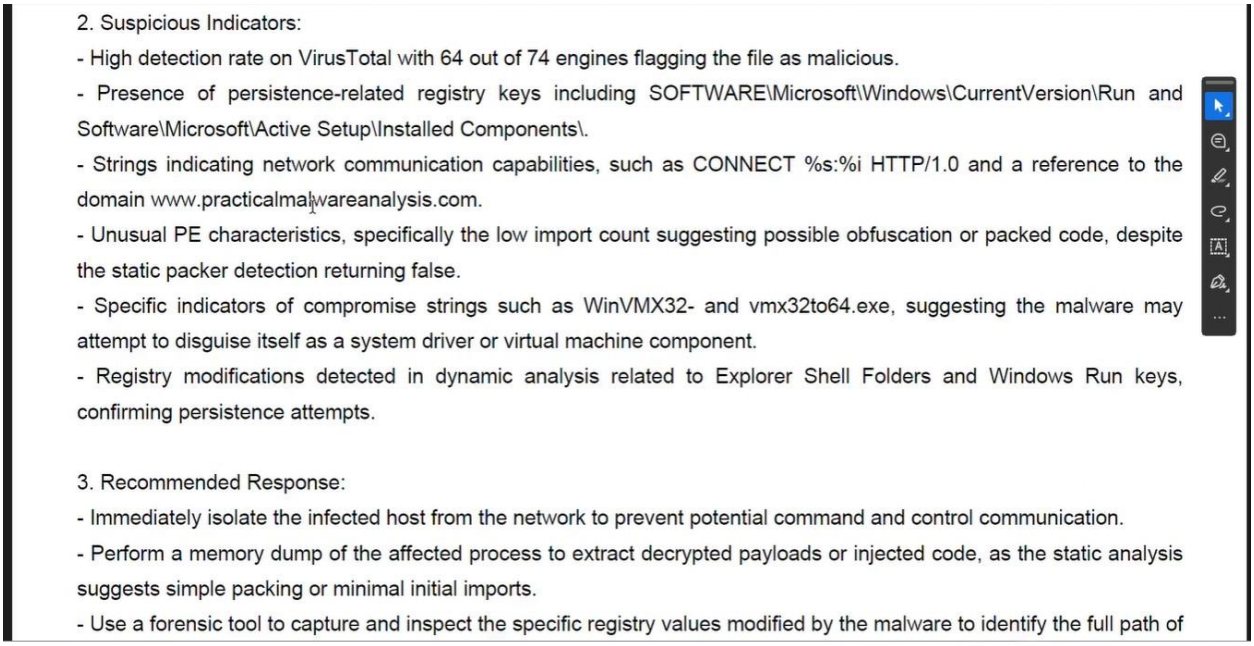
1. Executive Summary

Target File: Lab03-01.exe
Scan Date: 2026-05-09 23:20:40
Final Verdict: MALICIOUS
Risk Score: 7.2/100

2. AI Threat Intelligence Summary

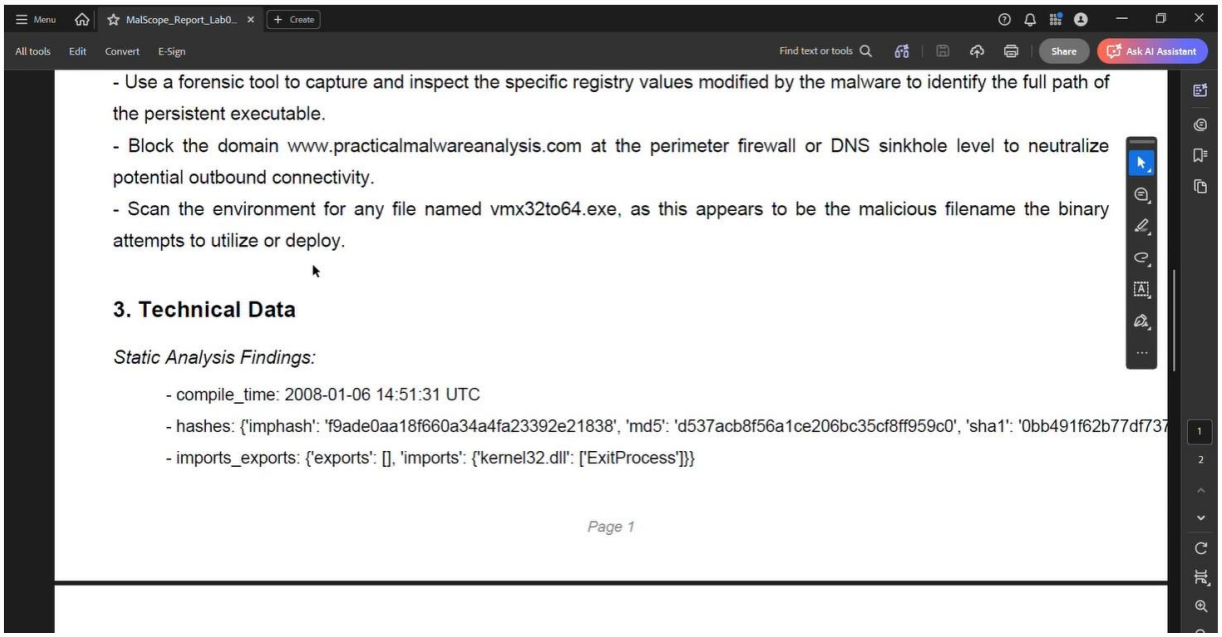
1. Threat Summary: Lab03-01.exe is a highly malicious executable likely functioning as a backdoor or a dropper designed for persistence. It attempts to maintain a foothold on the infected system by modifying critical Windows registry keys associated with autorun and system configuration.

2. Suspicious Indicators:
- High detection rate on VirusTotal with 64 out of 74 engines flagging the file as malicious.
- Presence of persistence-related registry keys including SOFTWARE\Microsoft\Windows\CurrentVersion\Run and



2. Suspicious Indicators:
- High detection rate on VirusTotal with 64 out of 74 engines flagging the file as malicious.
- Presence of persistence-related registry keys including SOFTWARE\Microsoft\Windows\CurrentVersion\Run and Software\Microsoft\Active Setup\Installed Components\.
- Strings indicating network communication capabilities, such as CONNECT %s:%i HTTP/1.0 and a reference to the domain www.practicalmalwareanalysis.com.
- Unusual PE characteristics, specifically the low import count suggesting possible obfuscation or packed code, despite the static packer detection returning false.
- Specific indicators of compromise strings such as WinVMX32- and vmx32to64.exe, suggesting the malware may attempt to disguise itself as a system driver or virtual machine component.
- Registry modifications detected in dynamic analysis related to Explorer Shell Folders and Windows Run keys, confirming persistence attempts.

3. Recommended Response:
- Immediately isolate the infected host from the network to prevent potential command and control communication.
- Perform a memory dump of the affected process to extract decrypted payloads or injected code, as the static analysis suggests simple packing or minimal initial imports.
- Use a forensic tool to capture and inspect the specific registry values modified by the malware to identify the full path of



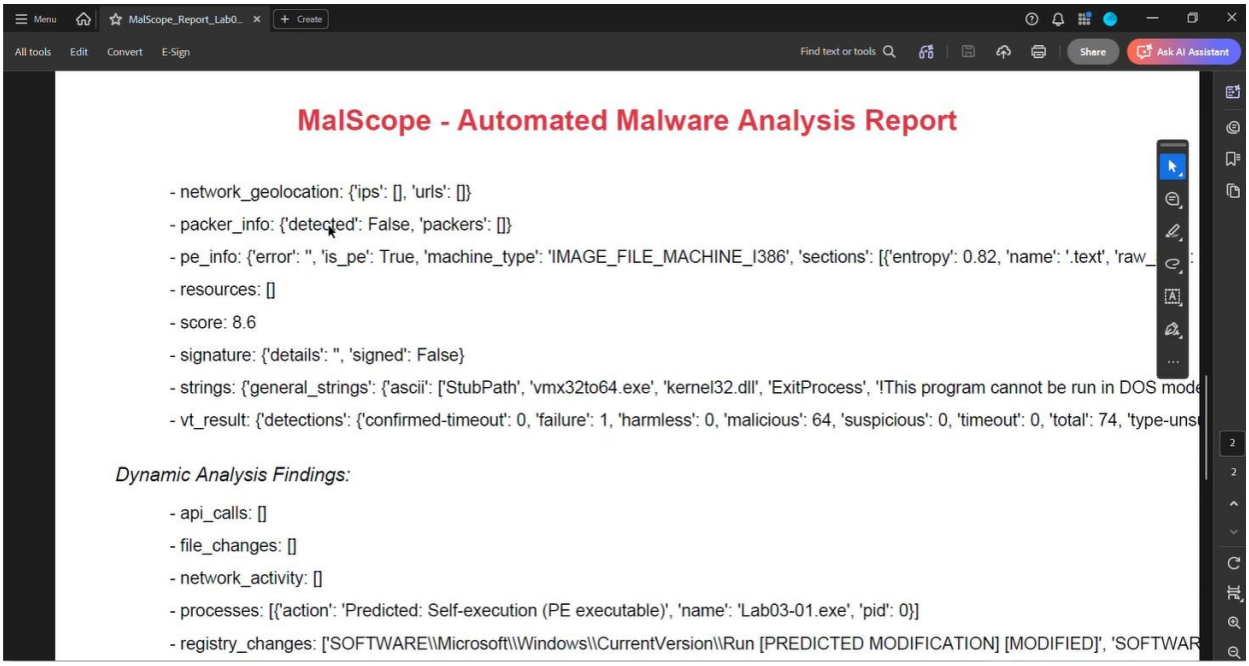
- Use a forensic tool to capture and inspect the specific registry values modified by the malware to identify the full path of the persistent executable.
- Block the domain www.practicalmalwareanalysis.com at the perimeter firewall or DNS sinkhole level to neutralize potential outbound connectivity.
- Scan the environment for any file named vmx32to64.exe, as this appears to be the malicious filename the binary attempts to utilize or deploy.

3. Technical Data

Static Analysis Findings:

- compile_time: 2008-01-06 14:51:31 UTC
- hashes: {'imphash': 'f9ade0aa18f660a34a4fa23392e21838', 'md5': 'd537acb8f56a1ce206bc35cf8ff959c0', 'sha1': '0bb491f62b77df737
- imports_exports: {'exports': [], 'imports': {'kernel32.dll': ['ExitProcess']}}

Page 1



MalScope - Automated Malware Analysis Report

- network_geolocation: {'ips': [], 'urls': []}
- packer_info: {'detected': False, 'packers': []}
- pe_info: {'error': '', 'is_pe': True, 'machine_type': 'IMAGE_FILE_MACHINE_I386', 'sections': [{'entropy': 0.82, 'name': '.text', 'raw_
- resources: []
- score: 8.6
- signature: {'details': '', 'signed': False}
- strings: {'general_strings': {'ascii': ['StubPath', 'vmx32to64.exe', 'kernel32.dll', 'ExitProcess', '!This program cannot be run in DOS mode
- vt_result: {'detections': {'confirmed-timeout': 0, 'failure': 1, 'harmless': 0, 'malicious': 64, 'suspicious': 0, 'timeout': 0, 'total': 74, 'type-unsu

Dynamic Analysis Findings:

- api_calls: []
- file_changes: []
- network_activity: []
- processes: [{'action': 'Predicted: Self-execution (PE executable)', 'name': 'Lab03-01.exe', 'pid': 0}]
- registry_changes: ['SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run [PREDICTED MODIFICATION] [MODIFIED]', 'SOFTWAR

13. Conclusion

MalScope demonstrates how a professional-grade malware analysis pipeline can be built with clean software engineering principles, industry-standard security methodologies, and modern AI integration. The system's modular architecture enables independent development of each analytical capability, standard JSON communication contracts enforce clean interfaces, and the signal-driven PyQt5 frontend ensures a responsive user experience despite computationally intensive backend operations.

The integration of five-hash fingerprinting, VirusTotal threat intelligence, PE structure analysis, behavioral prediction, IP geolocation, Hybrid Analysis sandbox submission, and Google Gemini AI interpretation into a single cohesive workflow reflects the multi-layered analysis approach required for effective malware triage in real-world SOC environments.

The 60/40 weighted scoring model provides a principled basis for risk quantification that balances the high confidence of crowd-sourced antivirus verdicts with the detection sensitivity of behavioral analysis. The three-tier verdict classification (Clean, Suspicious, Malicious) maps directly to standard incident response playbook actions, making MalScope's output immediately actionable for security analysts.

Looking forward, the planned integration of machine learning classification, enhanced AI heuristics, YARA rule matching, and real-time dynamic analysis positions MalScope as a platform capable of growing from an academic demonstration project into a production-ready security tool. The modular design ensures that each of these enhancements can be developed and integrated independently, following the same collaboration rules and interface contracts that governed the initial implementation.

MalScope succeeds as both a functional cybersecurity tool and as a demonstration of best practices in modular system design, API integration, and collaborative software development applicable to any complex, multi-component software project.

— End of Report —